

Adapter Design Pattern in Java

By **Arpit Mandliya**, 9 Oct 2012

★★★★★ 4.75 (4 votes)

[Download AdapterDesignPattern.zip - 6 KB](#)

Real Life Example

A very simple example is using phone charger. Suppose your mobile phone needs 9 Volts of supply to get charged but main supply is 220 V which is not what you require but it can be a source and you can have something that can get 9 V out of this 220 V and supply to your phone for charging. Now phone charger is acting as an adapter for you. So Basically it is making a relationship between two unrelated interfaces. This is how Adapter pattern works.

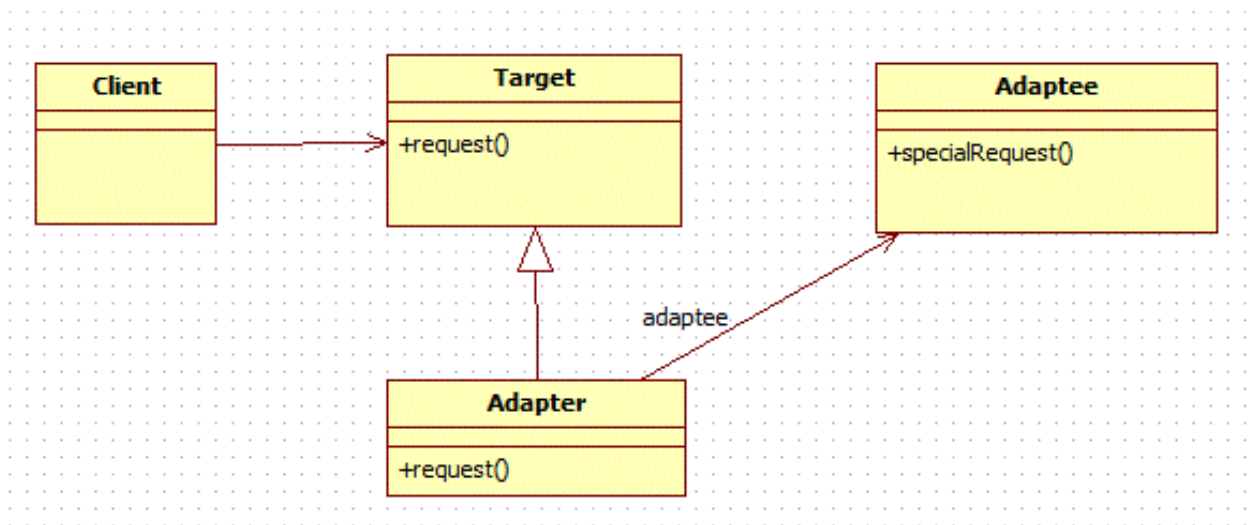
Intent

As described by Gof:

"Convert the interface of class into another interface clients expect. Adapter lets class work together that couldn't otherwise because of incompatible interfaces".

Also known as: Wrapper

UML Diagram of Generic Adapter Design Pattern



Elements

- **Target**
 - defines domains-specific interface client uses.
- **Client**

- collaborates with objects conforming to target interface.

- **Adaptee**

- defines existing interface that needs adapting

- **Adapter**

- adapts the interface of adaptee to target interface.

Workflow

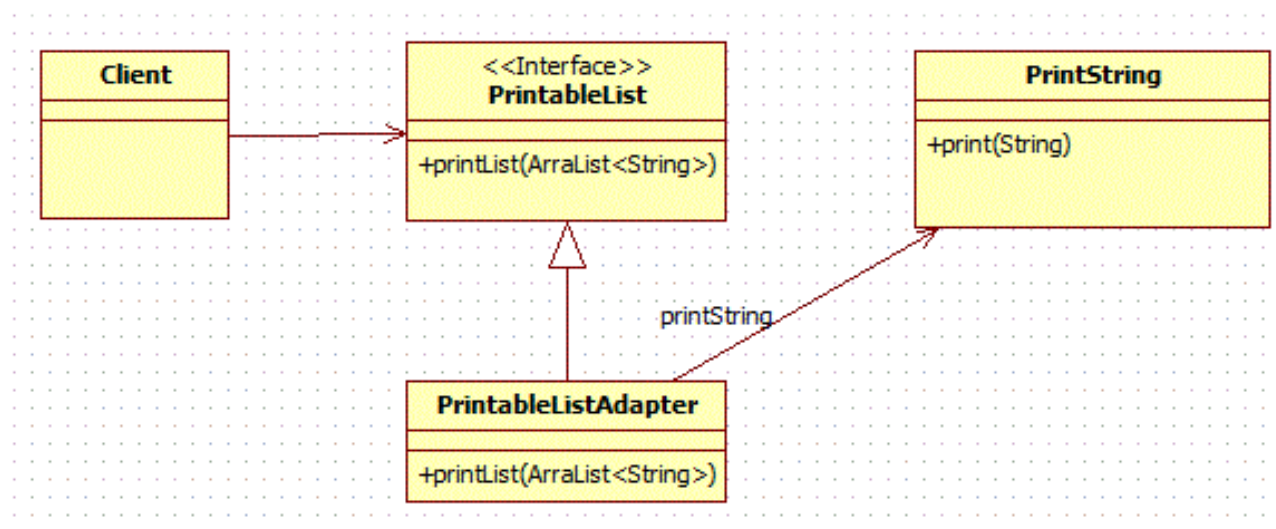
Clients call operations on the adapter instance. In turn adapter calls adaptee operations that carry out the request.

When to Use It

- You want to use existing class and its interface does not match the one you need.
- You want to create a reusable class that cooperates with unrelated or unforeseen classes that is, class that don't necessarily have compatible interfaces.

Example

UML Diagram for example



Comparing to above generic description of adapter pattern. My example includes following elements:

- PrintableList(Target)
- PrintString(Adaptee)
- PrintableListAdapter(Adapter)

Java Code for all above classes:

Consider that we have a third party library that provides print string functionality through **PrintString** class.

This is our **Adaptee**. I know this is silly assumption but lets go with it for now.

PrintString.java(Adaptee) :

```
package org.arpit.javapostsforlearning.designpatterns;

public class PrintString {

    public void print(String s)
    {
        System.out.println(s);
    }
}
```

Client deals with **ArrayList<String>** but not with string. We have provided a **PrintableList** interface that expects the client input. This is our **target**.

PrintableList.java(Target)

```
package org.arpit.javapostsforlearning.designpatterns;

import java.util.ArrayList;

public interface PrintableList {
    void printList(ArrayList<String> list);
}
```

Let's assume we can not change it now. Finally we have **PrintableListAdapter** class which will implement PrintableList interface and will deal with our adaptee class.

PrintableListAdapter.java(Adapter)

```
package org.arpit.javapostsforlearning.designpatterns;

import java.util.ArrayList;

public class PrintableListAdapter implements PrintableList{

    public void printList(ArrayList<String> list) {

        //Converting ArrayList<String> to String so that we can pass String to
        // adaptee class
        String listString = "";

        for (String s : list)
        {
            listString += s + "\t";
        }

        // instantiating adaptee class
        PrintString printString=new PrintString();
        ps.print(listString);
    }
}
```

AdapterDesignPatternMain.java

```
package org.arpit.javapostsforlearning.designpatterns;

import java.util.ArrayList;
public class AdapterDesignPatternMain {

    public static void main(String[] args)
    {
        ArrayList<String> list=new ArrayList<String>();
        list.add("one");
        list.add("two");
        list.add("three");
        PrintableList pl=new PrintableListAdapter();
        pl.printList(list);
    }
}
```

Output

```
one    two    three
```